

Competitive Exam Practice 1 — Answer Key

Foundations, Pointers & ADTs

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 13, 2026

- Q1 (C).** Structured programming’s point is readability of the *source*: using only sequence, selection, and iteration (never unrestricted `GOTO`) means a reader can trace the flow of control directly from the program text, without needing to look at what the compiler produced or worry about jumps landing anywhere. Confirmed against GeeksforGeeks and GATEOverflow (2026-08-13).
- Q2 (B).** The parameter list and the assignment `int m = a;` are *sequence*; the two `if` statements are *selection* (each independently may or may not execute); no `for/while` appears, so there is no *iteration* — comparing a fixed count of three values needs only sequence and selection, exactly as the two-mark version in the deck needed only sequence and selection for two marks.
- Q3 (B).** A local variable declared inside a function is automatic storage on the **stack**; `malloc` requests bytes from the **heap** at run time.
- Q4 TRUE.** A local variable lives in its function call’s stack frame and dies on return, while a heap block requested by `malloc` remains until the program releases it with `free` — the heap’s whole point is size and lifetime decided at run time.
- Q5 (C).** P1 returns the address of its local variable `x` (a dangling pointer — the stack frame dies when `g` returns), P2 dereferences the uninitialized pointer `px` before any `malloc`, while P3 requests a heap block first and is safe; hence only P1 and P2 cause problems.
- Q6 (D).** `pi == &i`, so `scanf` writes the entered value into `i` through the pointer; the printout is `i + 5`, exactly 5 more than the integer value entered, so the program compiles, runs, and prints “more than the integer value entered.”
- Q7 (D).** The first block from `malloc` is overwritten when `x = (int *)malloc(...)` executes inside the `if` — the old address is lost and can never be freed, so the code compiles but leaks memory; the final `free(x)` only releases the second block, and no dangling pointer is ever used.
- Q8 25.** `z = &a` makes `z` point at `a`; inside `foo`, `*p = x` writes the value of `b` (25) into `a` through the pointer, so the printed `a` is 25.
- Q9 TRUE.** `NULL` is the agreed “points to nothing” pointer value, and reading through it is undefined behaviour — in practice, a crash.
- Q10 (B).** `malloc(n)` asks for `n` bytes and returns their address, or `NULL` if the request cannot be met — which is why the result is always checked against `NULL` before use.

- Q11 (B).** `calloc` zero-initializes every byte of the block it hands back; `malloc` hands back the same number of bytes but leaves their contents unspecified (whatever was left over from a previous use of that memory) — same request size, different starting content.
- Q12 (B).** `realloc` preserves the block’s existing content up to the smaller of the old and new sizes, copying it to the new location when the block moves — so `q[0..2]` equal 10, 20, 30. It does *not* zero the newly added space (only `calloc` zeros space), so `q[3..5]` are unspecified. The original block at `p` is automatically released by `realloc` when it moves the block, so it must not be freed separately, and `realloc` gives no zero-initialization guarantee for the new slots.
- Q13 16.** `data` occupies 4 bytes (aligned at offset 0), `next` needs 8-byte alignment so 4 padding bytes follow `data`, and the pointer occupies bytes 8–15; total $4 + 4 + 8 = 16$ bytes, which is also a multiple of the 8-byte alignment of the struct.
- Q14 (C).** An ADT is defined by the operations available on it — the interface — and nothing about its internal representation is exposed or usable directly; this is exactly the “set of values and set of operations” framing (Sedgewick, p.64) and the “declaration of data plus declaration of operations” framing (Karumanchi, p.18) the course uses. It is not the same as an abstract class (A) and instantiability (B) is a separate, unrelated property. Confirmed against GeeksforGeeks and GATEOverflow (2026-08-13).
- Q15 (B).** The bag’s contract names the operations (`insert`, `member`, `size`) — a set of values and a set of operations — and deliberately withholds the storage scheme; “dynamic array” and “linked list” are two interchangeable *implementations* of that same contract, and code and costs are implementation details, not contract.